NIT3112 Lab Assessment 1

QuTech subscription onboarding

Application report and administrator user manual

Justin Lee
Student ID S8115784
Victoria University
19 October 2026

Purpose

This report explains the QuTech C# console application, the classes and calculation rules used to create client subscriptions, and the observed results of execution tests. The application lists four plans, accepts multiple valid subscriptions, applies a 15% student discount and displays a session summary.

The solution is named JustinLee-QuTech. The administrator is Justin Lee and the discount code is JUST. The application passed all 12 recorded black-box cases; the five-entry scenario confirms the example prices and all required summary fields.

Reading guide

Section	Contents
1	Scope and operation
2	Classes and calculation design
3	Input validation and control flow
4	Illustrated administrator manual
5	Test design and observed results
6	Assessment mapping and limitations

The evidence uses fictional demonstration entries. Subscriptions are retained in memory for the current process only.

1 Scope and operation

QuTech administrators onboard a business by selecting a plan, entering the number of users and choosing whether to apply the student discount. A successful entry immediately shows its monthly price and is available in the summary until the program exits.

Run the submitted application

1. Install the .NET 10 SDK. Verification used SDK 10.0.401 on macOS.
2. Extract JustinLee-Lab1.zip and open the JustinLee-QuTech folder.
3. Run the following command in that folder, or open JustinLee-QuTech.sln in a compatible Visual Studio installation.

```
dotnet run --project JustinLee-QuTech/JustinLee-QuTech.csproj
```

4. Choose A to list plans, B to create a subscription, C to view all subscriptions, or D to quit.
5. Enter JUST for the 15% discount or No to continue without it. Menu letters and discount responses are case-insensitive.

Plan catalogue

ID	Plan	Monthly price per user
0	Startup	$10.00
1	Standard	$20.00
2	Premium	$30.00
3	Enterprise	$40.00

Scope boundaries

The identity banner is hard-coded as permitted by the brief; no credential prompt is required. No database, web server or third-party NuGet packages are used. The supplied scenario mentions optional modules, but provides no module prices or selection specification; this application implements the detailed plan and user workflow.

The project excludes generated bin and obj directories. A compatible SDK rebuilds these files. Subscriptions intentionally do not survive exit, matching the session-based console workflow.

2 Classes and calculation design

Responsibilities and constructors

Program.Main constructs AdminPortal and calls Run. AdminPortal owns the plan array and List<Subscription>, reads inputs, repeats the menu and displays results. Separate ShowPlans, CreateSubscription, ShowSubscriptions, Read and ReadInteger methods keep each operation focused.

Plan stores immutable Id, Name and MonthlyPrice properties. Its constructor creates the four catalogue objects. Subscription stores BusinessName, Plan, Users and HasDiscount. Its constructor validates the business name and user count, then fixes the values for the saved entry. Each successful B operation creates a new object and adds it to the list.

Variables and types

Integer values represent IDs and user counts; string values hold names and menu responses; bool values track the active loop and discount state. Decimal prices and the 0.15m constant preserve exact base-10 arithmetic. Arrays hold the fixed catalogue and a generic list holds the growing subscription collection.

Monthly calculation

Subtotal = monthly plan price × users. DiscountAmount = Subtotal × 0.15 when JUST is supplied, otherwise zero. TotalPrice = Subtotal − DiscountAmount. PricePerUser applies the same percentage to the plan unit price. Monetary output uses invariant two-decimal formatting with a dollar sign.

Example	Subtotal	Discount	Total
Standard × 20, No	$400.00	$0.00	$400.00
Premium × 25, JUST	$750.00	$112.50	$637.50
Enterprise × 100, JUST	$4000.00	$600.00	$3400.00

The supplied example of Standard with 20 users totals $400.00; this establishes that the listed plan amount is multiplied by the user count. All four catalogue prices and the fixed discount produce exact cents, so no additional rounding rule is needed for these inputs.

Project structure

JustinLee-QuTech.sln contains the console project. Program.cs contains the entry point, Plan, Subscription and AdminPortal in the JustinLee_QuTech namespace. tests/run_tests.py builds and executes the program; results.json retains the input sequences, expected output and observed stdout. evidence contains the test output captures.

3 Input validation and control flow

Run uses a while loop and switch to handle A, B, C and D. Invalid options report the available choices and return to the menu. ShowPlans and ShowSubscriptions use foreach loops to display the relevant objects. ReadInteger loops until int.TryParse succeeds and the value lies within its inclusive limits.

Input	Accepted values	Failure behaviour
Menu	A, B, C, D	Explain choices and repeat
Business name	1–60 printable characters after trimming	Repeat name prompt
Plan ID	Whole number 0–3	Repeat plan prompt
Users	Whole number 1–100	Repeat user prompt
Discount	JUST or No, any letter case	Repeat discount prompt
End of input	EOF at any prompt	Exit cleanly; omit unfinished entry

Blank names, control characters, out-of-range values, decimals, text and integer overflow cannot become saved entries. Read trims surrounding whitespace. A mistaken discount is rejected explicitly so that the administrator can correct it before saving.

CreateSubscription adds a record only after every field is valid. The Subscription constructor independently enforces the business name and user count rules. If input ends midway, the exception is handled by Run and the unfinished entry is not added.

Summary behaviour

Option C displays Business name, Plan, Users, Total Price and Discount Y or N for every saved subscription. The business-name column expands to the longest stored name. The summary also shows a record count and combined monthly total. An empty list displays guidance to create a subscription.

Program readability

PascalCase names identify types, methods and properties; camelCase identifies local variables and private fields. Comments explain money types, EOF handling and the save boundary. The source uses consistent indentation and separates input, calculation and output responsibilities through methods and classes.

4 Administrator manual

These screenshots were captured while operating the running .NET console application through an xterm browser terminal connected to a local pseudo-terminal (PTY). Keyboard input is forwarded to the process and output is displayed live. The images show entered values and console responses; they are not replayed test logs.

Start the console application and enter A. The live screen shows Justin Lee as Administrator and all four plan IDs and monthly prices.

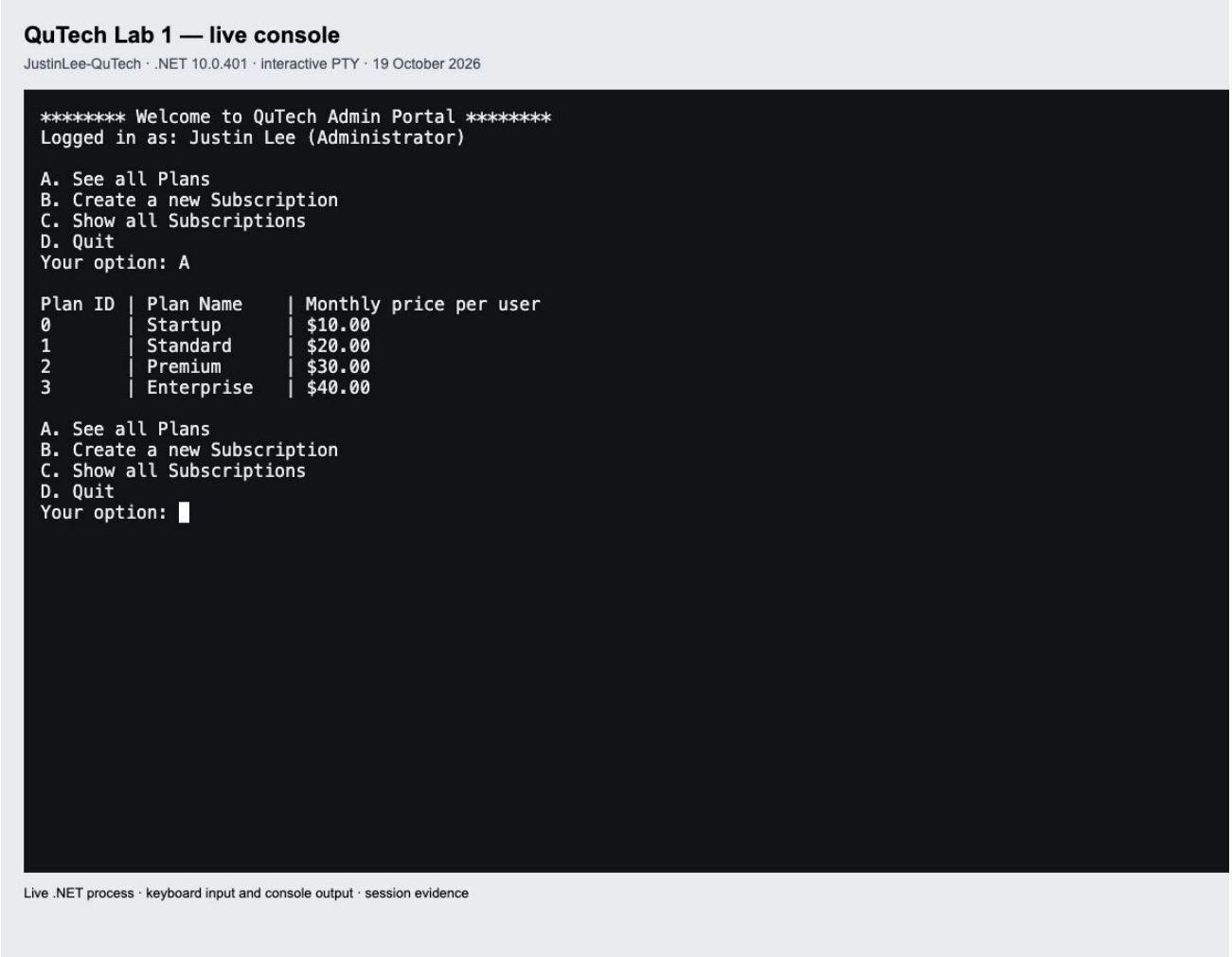


Figure 1. Live console plan listing with the actual A input.

All five subscription entries and the summary were captured in one uninterrupted process. The supplementary negative-input check follows the five-entry summary. Raw terminal output and keyboard events accompany the source ZIP.

4 Create Standard without a discount

Enter B, Woolies, plan 1, 20 users and No. The console accepts the entry and confirms $20.00 per user and a total monthly price of $400.00.

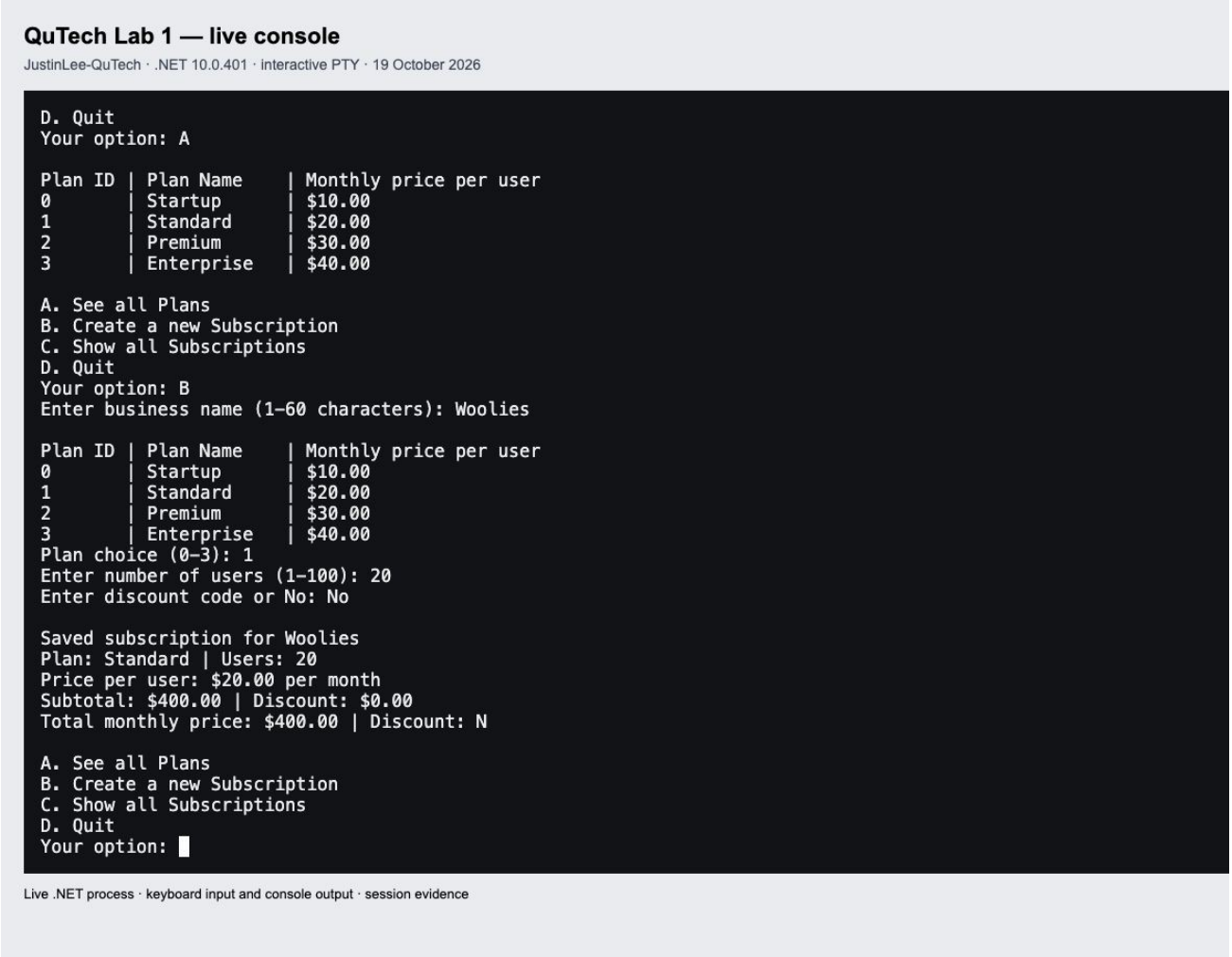


Figure 2. Entry 1 shows the prompts, entered values and saved confirmation.

4 Apply the student discount

Enter B, Coles, plan 2, 25 users and JUST. Premium has a $750.00 subtotal; the 15% discount is $112.50 and the final monthly total is $637.50.

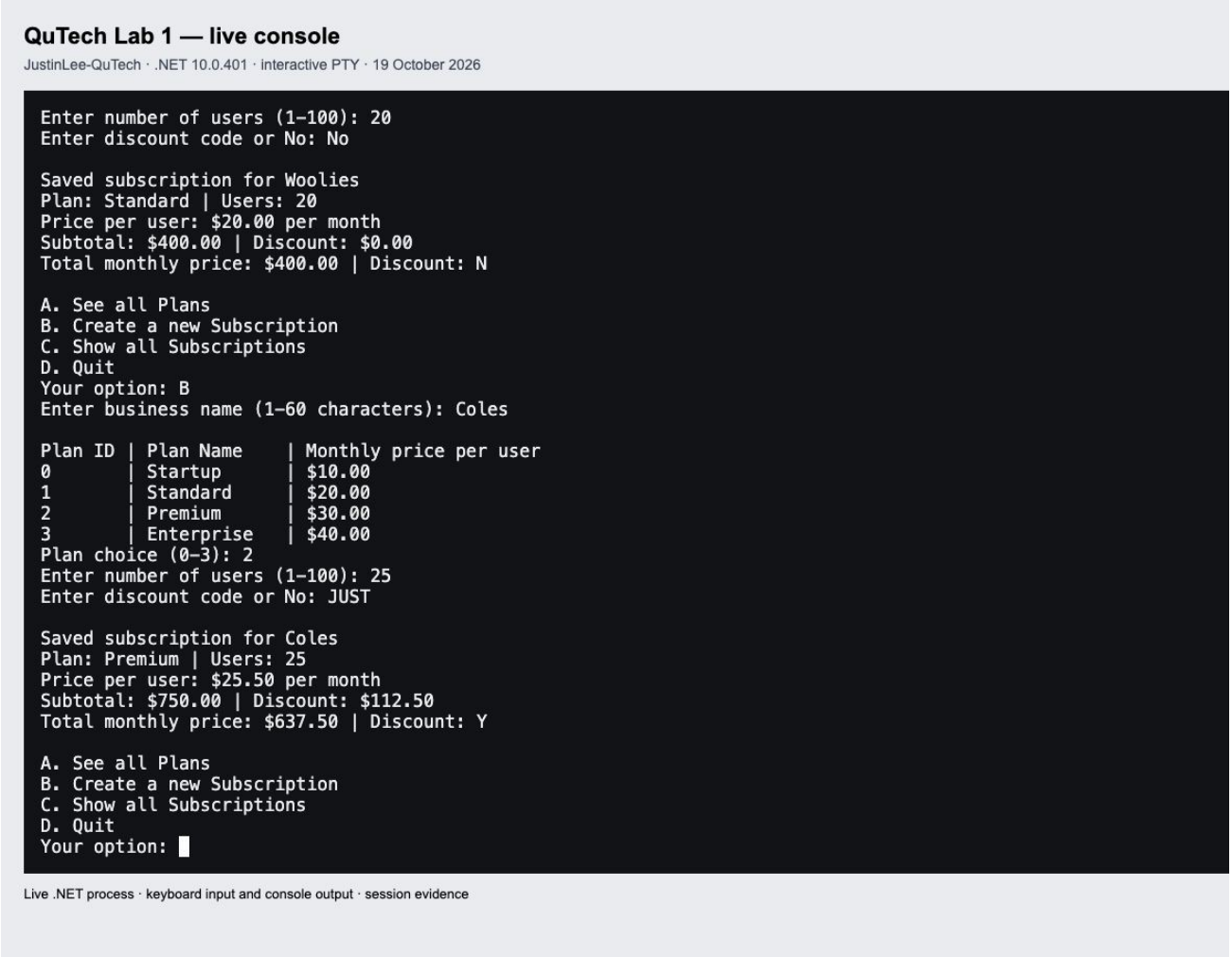


Figure 3. Entry 2 demonstrates the JUST code and correct discounted price.

4 Test the minimum user count

Enter B, Harbour Studio, plan 0, 1 user and No. The lower accepted boundary gives a Startup monthly total of $10.00.

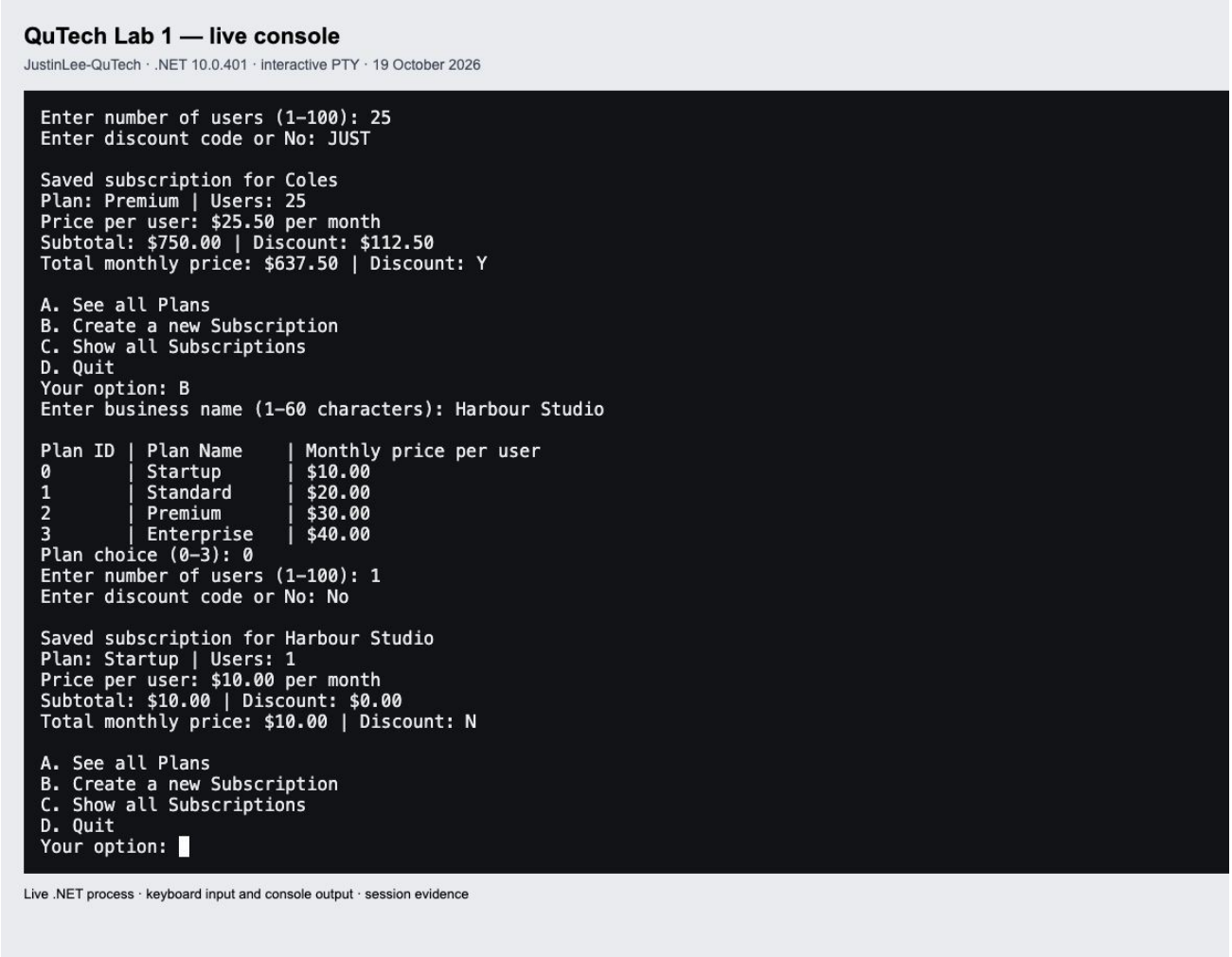


Figure 4. Entry 3 confirms the lower boundary of one user.

4 Test the maximum user count

Enter B, Summit Logistics, plan 3, 100 users and JUST. Enterprise produces a $4000.00 subtotal, a $600.00 discount and a $3400.00 monthly total.

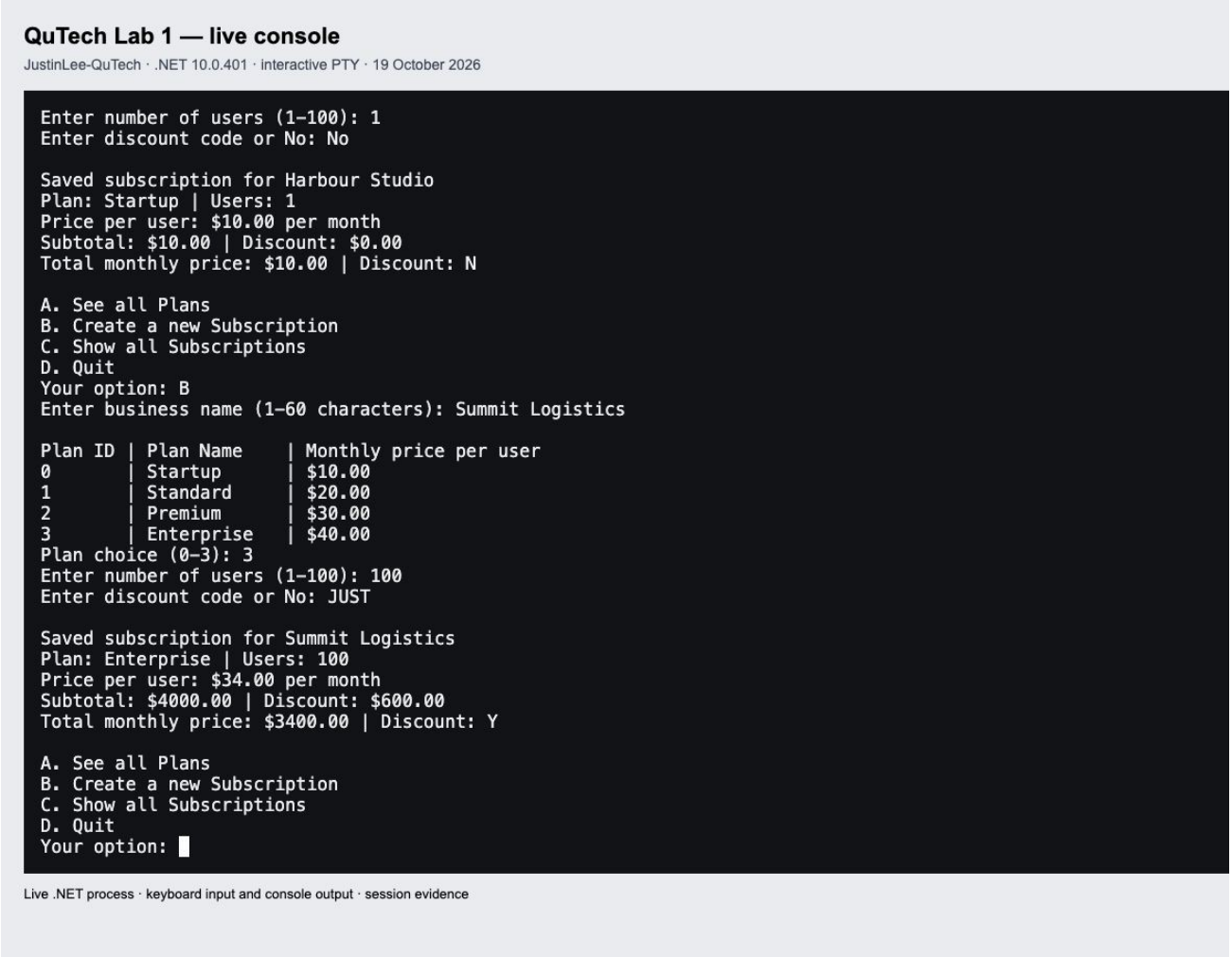


Figure 5. Entry 4 confirms the upper boundary of 100 users.

4 Accept a lowercase discount code

Enter B, Greenfield Dental, plan 1, 10 users and just. The lowercase code is accepted and reduces the $200.00 subtotal to $170.00.

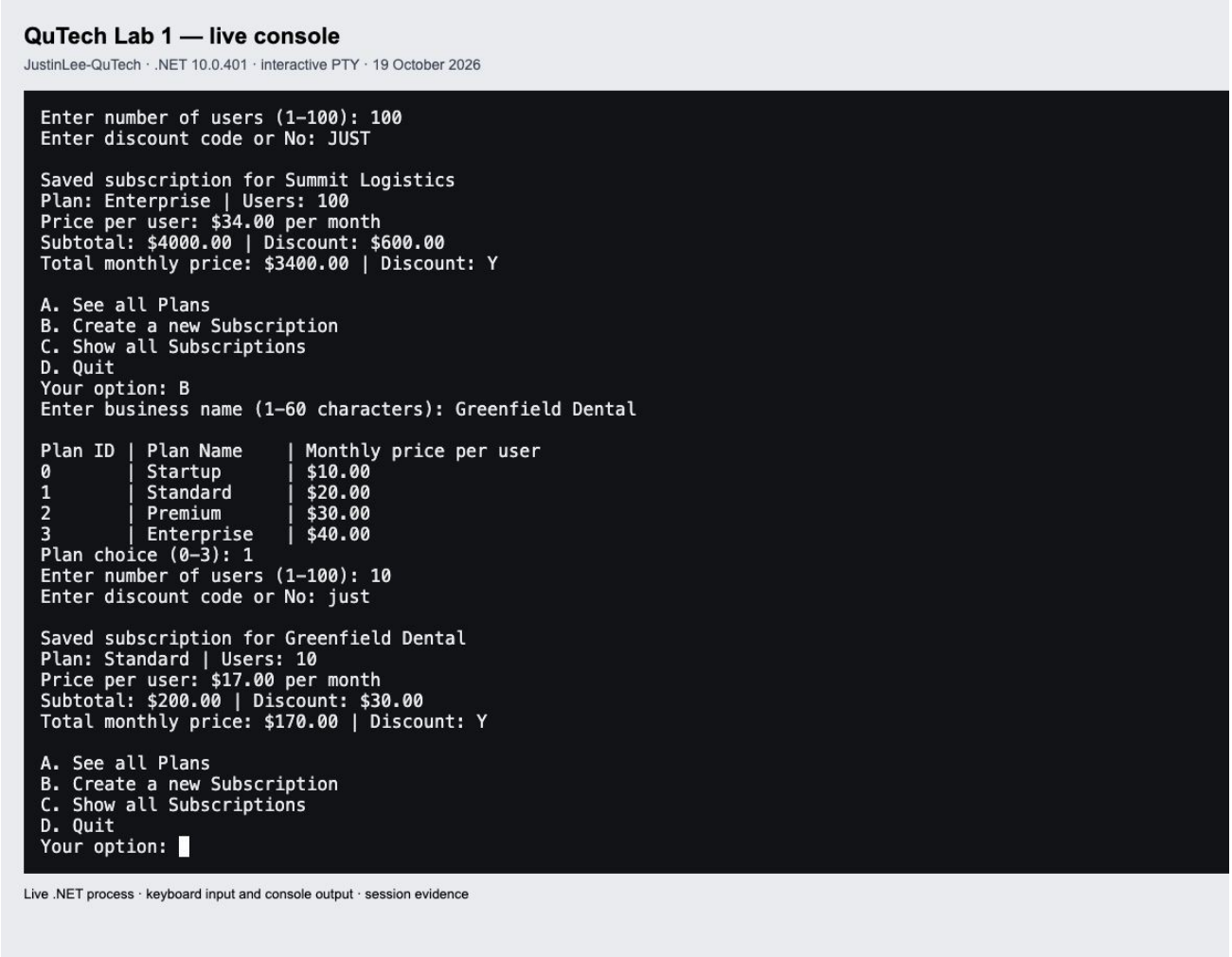


Figure 6. Entry 5 confirms case-insensitive discount input.

4 Review all five subscriptions

Enter C after the five entries. The same console process displays each business name, plan, user count, total price and Y or N discount flag. Exactly five subscriptions are listed, with a combined monthly total of $4617.50.

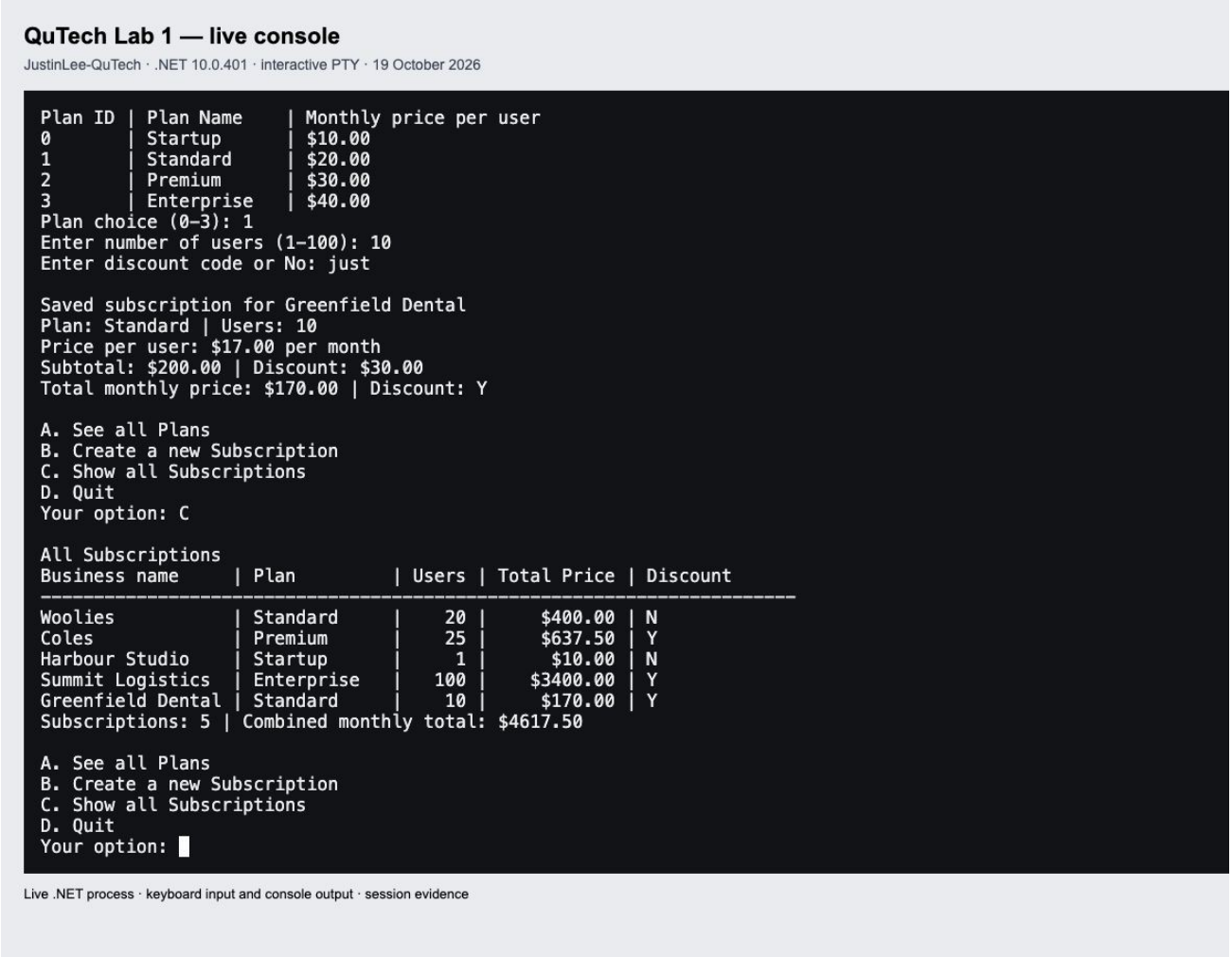


Figure 7. Live five-entry summary with the actual C input and all required columns.

4 Reject invalid input

After capturing the five-entry summary, begin a supplementary validation entry. Enter ?, then B, a blank name, Validation Check, plan 4 then 0, and user values 0, 101 and abc. Each invalid field is rejected and the console continues requesting a valid value.

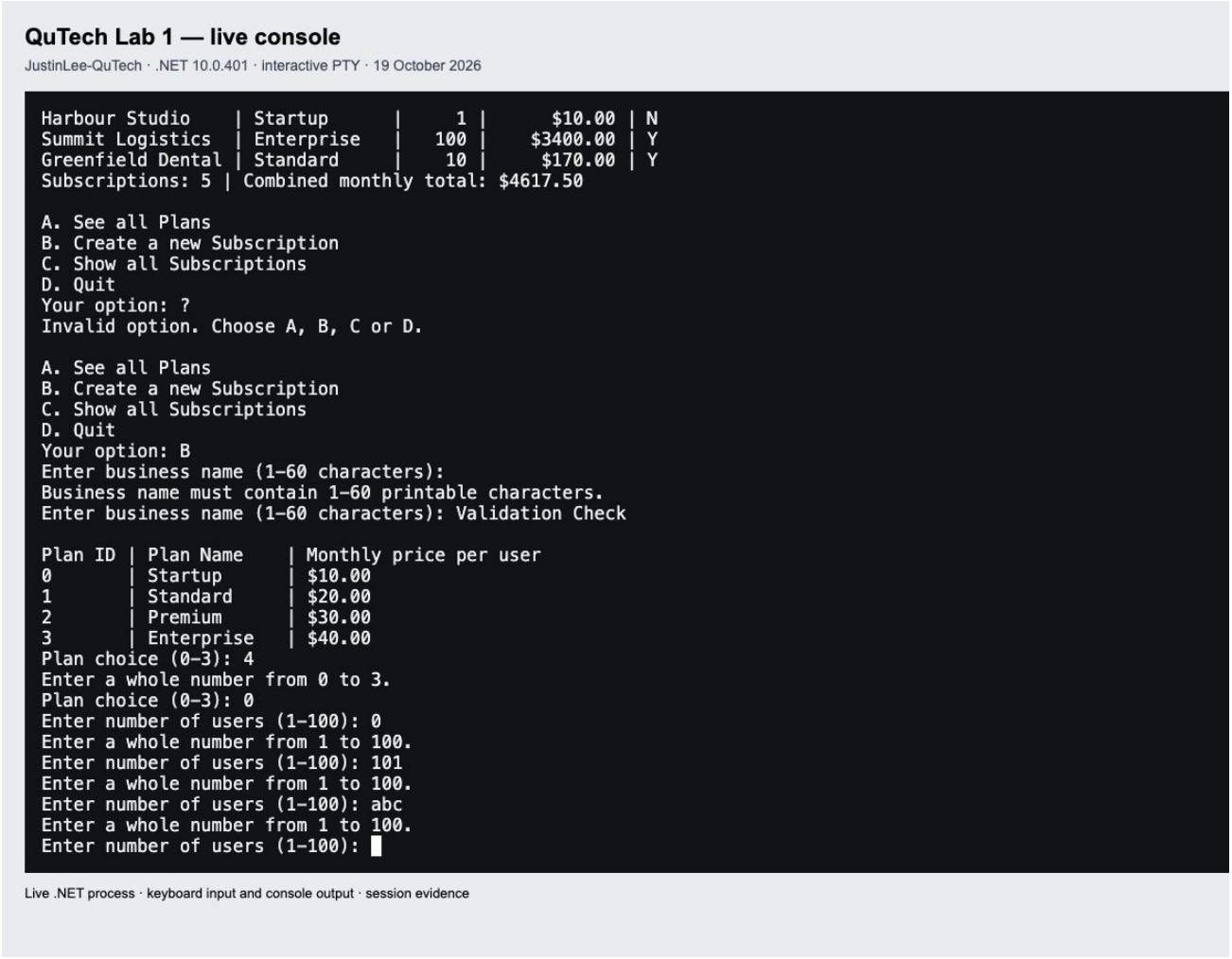


Figure 8. Actual invalid menu, blank name, invalid plan and invalid user inputs.

4 Correct errors and save

Continue the supplementary entry with 1 user. Enter BAD as the discount code; the console rejects it. Enter No to save one valid Startup subscription for Validation Check at $10.00. This is a sixth demonstration entry created after the five-entry summary.

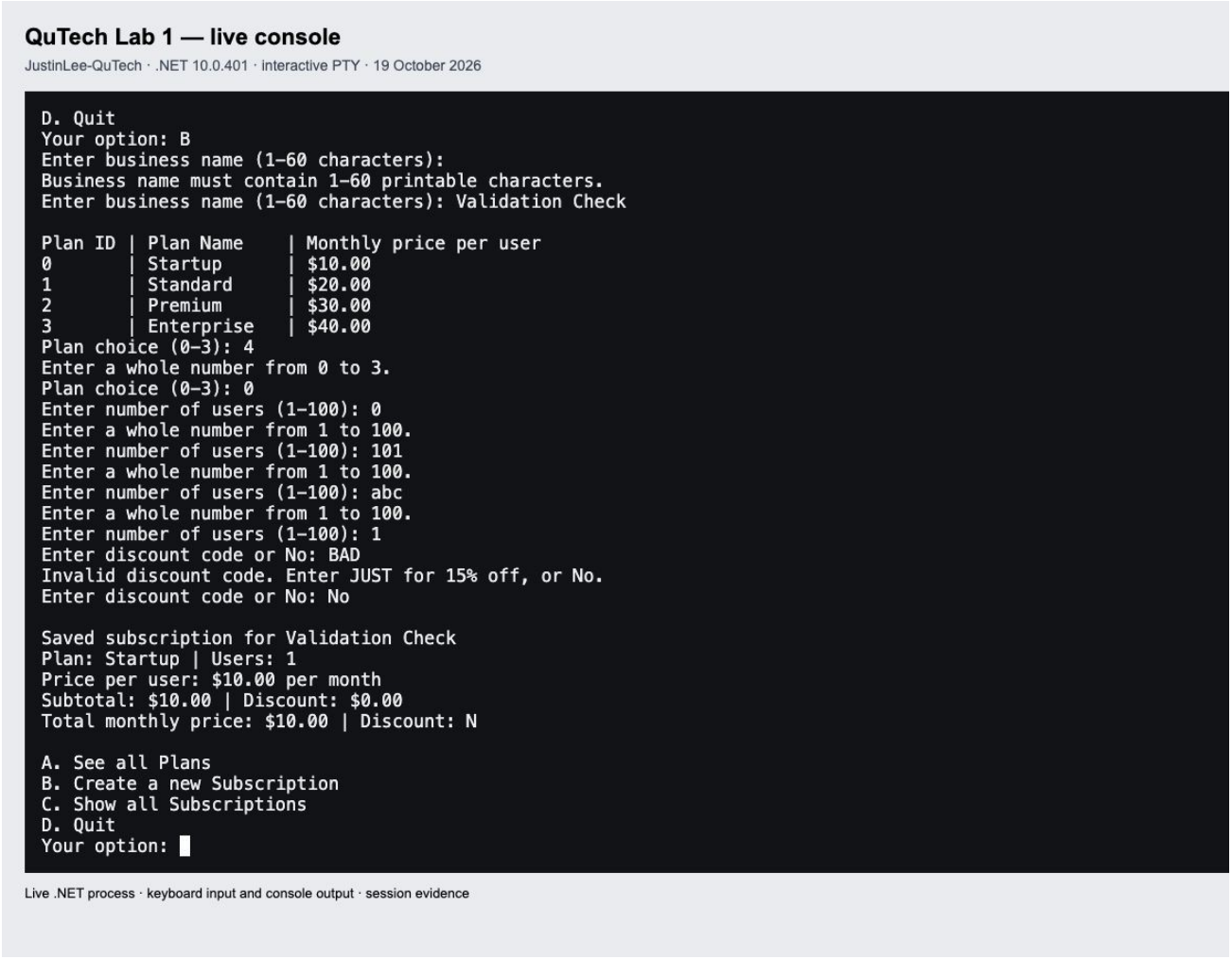


Figure 9. Invalid discount rejection and successful correction without a crash.

4 Quit the console

Enter D. The console displays the goodbye message and the .NET process exits normally. Session-only records are discarded on exit.

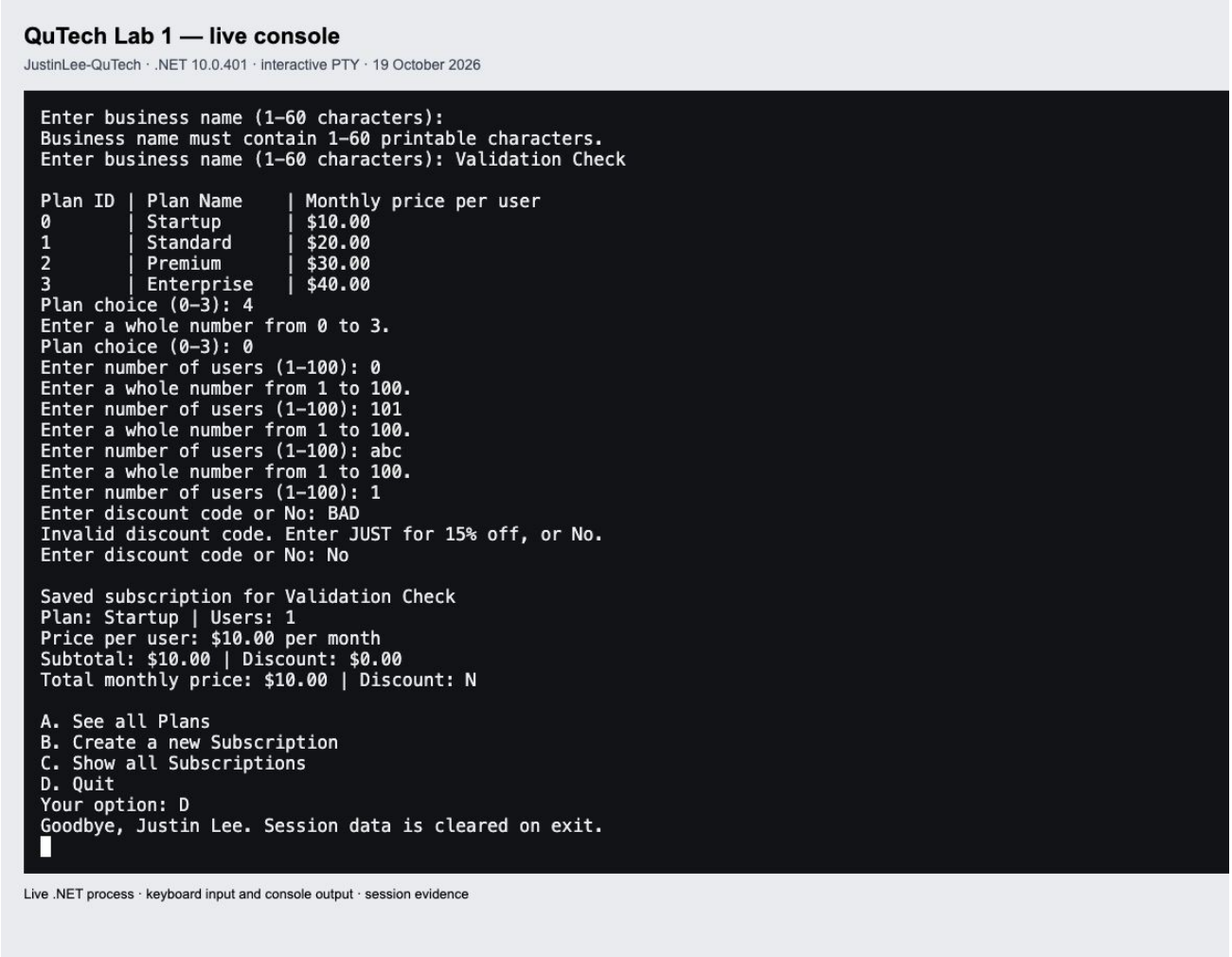


Figure 10. Actual D input and successful normal termination.

5 Test design and observed results

The black-box runner builds the project and launches a fresh .NET process for each test. It submits known input lines, checks expected output fragments and process success, and writes the actual stdout into JSON. The five-entry scenario additionally checks each summary row for the business name, total and discount flag. All 12 cases passed. Build verification reported zero warnings and zero errors.

ID	Procedure and expected outcome	Observed
01	A then D: banner and four priced plans	PASS — banner; four plan prices; normal exit
02	C on fresh start: empty-list guidance	PASS — empty-list guidance displayed
03	Create five entries then C: five rows, $4617.50 combined	PASS — 5 correct rows; $4617.50 total
04	Invalid menu, name, plan, users and code; recover to one $400 record	PASS — errors reported; one $400.00 entry
05	Startup, 100 users, No: $1000.00	PASS — $1000.00
06	Standard, 100 users, No: $2000.00	PASS — $2000.00
07	Premium, 100 users, No: $3000.00	PASS — $3000.00
08	Enterprise, 100 users, No: $4000.00	PASS — $4000.00
09	Trimmed lowercase b and just, one Startup user: $8.50	PASS — $8.50; discount Y
10	61-character name rejected; corrected name saved	PASS — long name rejected; Valid saved
11	EOF during an entry: clean exit, no saved entry	PASS — clean exit; unfinished entry absent
12	Fresh process: previous session data absent	PASS — previous entries absent

Reproduce the results

From the extracted solution directory, run python3 tests/run_tests.py. Python is optional test tooling; the application itself requires only the .NET SDK. The test runner uses subprocess input so stdout does not echo the typed values. Each test record therefore includes its input separately from the captured output.

The checks establish the observed console behaviour on .NET SDK 10.0.401. They do not claim interactive screen-reader, cross-platform terminal or Visual Studio UI testing. Five distinct entries cover all plans, both discount states and both user-count boundaries.

6 Assessment mapping and limitations

Criterion	Marks	Evidence
Variables	5	Section 2 and typed source properties
Constructors objects and classes	10	Plan, Subscription and AdminPortal
Loops and conditions	5	Section 3; while, switch and foreach
Structure and naming	10	Named solution, namespace and focused methods
Methods	5	Run, input readers, create and display methods
Input and output	5	Section 4 and execution evidence
Input validation	10	Section 3; test cases 04 and 09–11
Price calculation	5	Section 2; cases 03 and 05–09
Summary table	15	Figure 7 and five-row assertions
Logic and justification	10	Calculation rationale and session workflow
Spacing and indentation	5	Consistently formatted C# source
Code comments	5	Money, EOF and save-boundary comments
Report and test cases	10	Five entry captures and 12 recorded cases

Limitations and references

Records exist only in memory. The console identity banner is not authentication. The application does not calculate tax, annual billing or external payments. The business-name length limit is an implementation usability choice; duplicate business names remain allowed as separate subscriptions.

Teaching source: NIT3112 Advanced Web Application Development, Lab Assessment 1, supplied five-page brief. Pages 1–4 specify the workflow, plan prices, 15% discount, five-entry evidence and Word report plus source ZIP. Page 5 supplies the 100-mark rubric. Report layout follows the previous Justin Lee Lab Assessment 3 report; Lab 1 source and results were produced independently for this submission.

Student: Justin Lee, S8115784. Verification date: 19 October 2026.